

SMART INDIA HACKATHON 2026

MPLADS RAKSHAK

An AI-powered monitoring, risk-detection and accountability platform for the Members of Parliament Local Area Development Scheme.

Problem Statement ID

26102

Theme · PS Category

Smart Automation · Software

Team

The-Lolgorithms · Team ID 146579

Table of Contents

Table of Contents.....	
1. Introduction & Problem Statement	1
1.1 The Scheme	1
1.2 The Problem.....	1
1.3 Our Response: MPLADS Rakshak	1
2. Background & Rationale	2
2.1 Scale and Fragmentation	2
2.2 Where Manual Oversight Falls Short.....	2
2.3 Why AI Actually Helps Here	2
3. Review of Existing Systems.....	2-3
4. System Architecture	3-4
4.1 End-to-End Flow	3
4.2 Why React + Vite for the Frontend	4
5. Backend Engineering: Core Pipelines	4-7
6. Risk Scoring Framework.....	8
6.1 Weighted Composite Score	8
6.2 Risk Bands.....	8
6.3 Explainability.....	8
7. Citizen Grievance Filing & Resolution	9-10
7.1 Where a Report Is Designed to Go.....	9
7.2 Issue Categories and Statutory Risk	9
7.3 Designed Resolution Workflow	10
8. Multi-Citizen Aggregation & Escalation	10-11
8.1 Planned Severity Escalation	10
8.2 Planned Financial Safeguards and Anti-Spam Controls	10
8.3 Unified Resolution	11
9. Infrastructure & Deployment	11-12
9.1 Photo Storage.....	11
10. Security, RBAC & Governance.....	12-13
10.1 Role-Based Access Control	12
10.2 Platform-Wide Controls.....	13
11. Stakeholder Dashboards	13
12. Implementation Roadmap	13
13. Success Metrics	14
14. Conclusion & Future Scope.....	14

O1

Introduction & Problem Statement

The Scheme

The Members of Parliament Local Area Development Scheme (MPLADS) lets Members of Parliament recommend developmental works in their constituencies — the idea being that local MPs know their areas well enough to push for durable community assets and basic civic amenities that might otherwise get overlooked. Funds go out to district authorities every year and get executed by a long list of implementing agencies, which is how you end up with thousands of works running at once: roads, drinking-water schemes, community halls, sanitation projects, and more, spread across every constituency in the country.

The Problem

Problem Statement 26102 asks for an AI-powered system that can catch anomalies, fraud, and inefficiency in how MPLADS is actually implemented. That's a reasonable ask once you look at the numbers — hundreds of MPs, thousands of implementing agencies, lakhs of individual works. No manual, sample-based audit process was ever going to keep pace with that. In practice, this means things like inflated costs, duplicate works, and ghost projects tend to surface only during periodic audits, often years after the money has already gone out the door.

Our Response: MPLADS Rakshak

What we've built is a full-stack platform — a React/Vite frontend, a FastAPI backend, a rule-based and statistical anomaly engine, a retrieval-augmented compliance engine, a computer-vision forensics layer, and a citizen-facing reporting channel. The goal isn't to replace anyone's authority; it's to shift MPLADS monitoring from sparse, after-the-fact sampling to something closer to continuous, risk-ranked oversight, while leaving the actual decisions where they belong — with District Collectors, State Nodal Officers, and the Ministry.

5 roles, 1 platform

Citizen, Contractor, District Authority, Member of Parliament, and Ministry Admin each get a purpose-built, role-gated view of the same underlying data.

02

Background & Rationale

Scale and Fragmentation

MPLADS moves thousands of crores of rupees every year across more than five hundred parliamentary constituencies. Each individual work goes through recommendation, sanction, fund release, tendering, execution, inspection, and completion — and at every one of those stages, someone somewhere is generating a record. The trouble is those records live in different district offices, different implementing agencies, and different state departments, with very little cross-referencing between them.

Where Manual Oversight Falls Short

- Monitoring tends to be retrospective — problems usually surface during an audit conducted years after the money was spent.
- There simply aren't enough inspectors to physically visit more than a fraction of active works nationwide.
- Comparing cost, vendor history, and geography across thousands of works by hand isn't realistic for any team.
- Photos submitted as completion evidence are rarely checked for whether they're genuine, reused, or even taken at the right place.
- There's no easy way today to check whether a work's cost or timeline looks reasonable next to similar works elsewhere.

Why AI Actually Helps Here

This is exactly the kind of problem machine learning and geospatial analysis are good at — sifting through more structured and unstructured data than any team could review by hand, while picking up on what's normal for a given category of work, region, or agency. None of this is meant to replace statutory inspection. It's there to point limited investigative attention at the handful of works that actually look statistically or contextually off, instead of leaving that discovery to chance.

03

Review of Existing Systems

MPLADS already has some digital infrastructure behind it — the MIS run by MoSPI, the e-SAKSHI citizen transparency portal, and the CPWD Schedule of Rates that's used as a construction-cost reference. All of these are good at what they were built for: keeping records and disclosing information publicly. What none of them do is detection. There's no cross-work comparison, no duplicate-work flagging, no contractor network analysis, and no automated check on whether a geo-tagged photo actually matches where it claims to be from.

Existing System	Primary Function	Gap MPLADS Rakshak Fills
MPLADS MIS (MoSPI)	Central record of sanctions, releases, works	No automated anomaly, duplicate, or risk scoring
e-SAKSHI Portal	Public disclosure of work-level status	No cross-work pattern analysis or fraud-risk scoring
CPWD Schedule of Rates	Reference cost baseline	Not programmatically compared to tender/BOQ costs at scale
Manual / periodic audits	Statutory compliance verification	Reactive, sample-based, delayed by months or years

04 System Architecture

We built MPLADS Rakshak as four layers: a role-based React frontend, a FastAPI backend exposing REST routers, a set of AI services (the RAG compliance engine, the anomaly engine, and the image verifier), and a data layer that pairs a relational store with a vector store. Each layer can be deployed on its own, which in practice just means we can improve the detection logic without ever having to touch the presentation layer, and the other way around.

End-to-End Flow



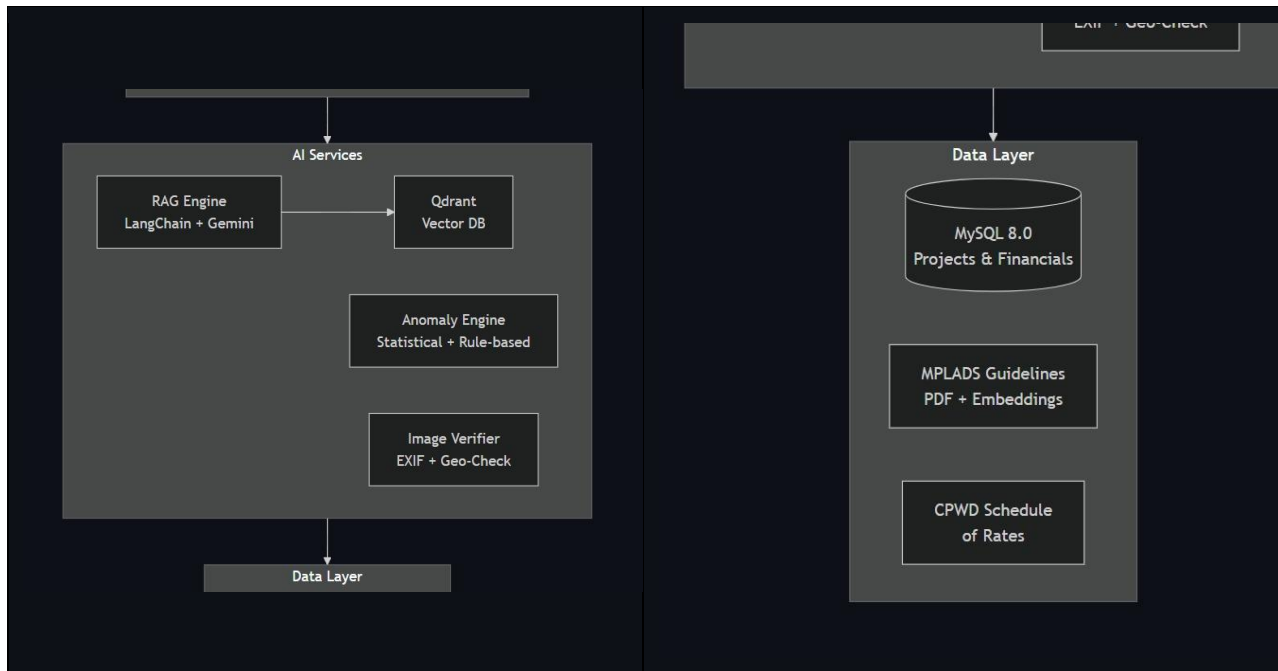


Figure 1 — Request flow: role-based frontend → FastAPI routers → AI services (RAG, anomaly, image forensics) → data layer (MySQL, guideline embeddings, CPWD rates).

Why React + Vite for the Frontend

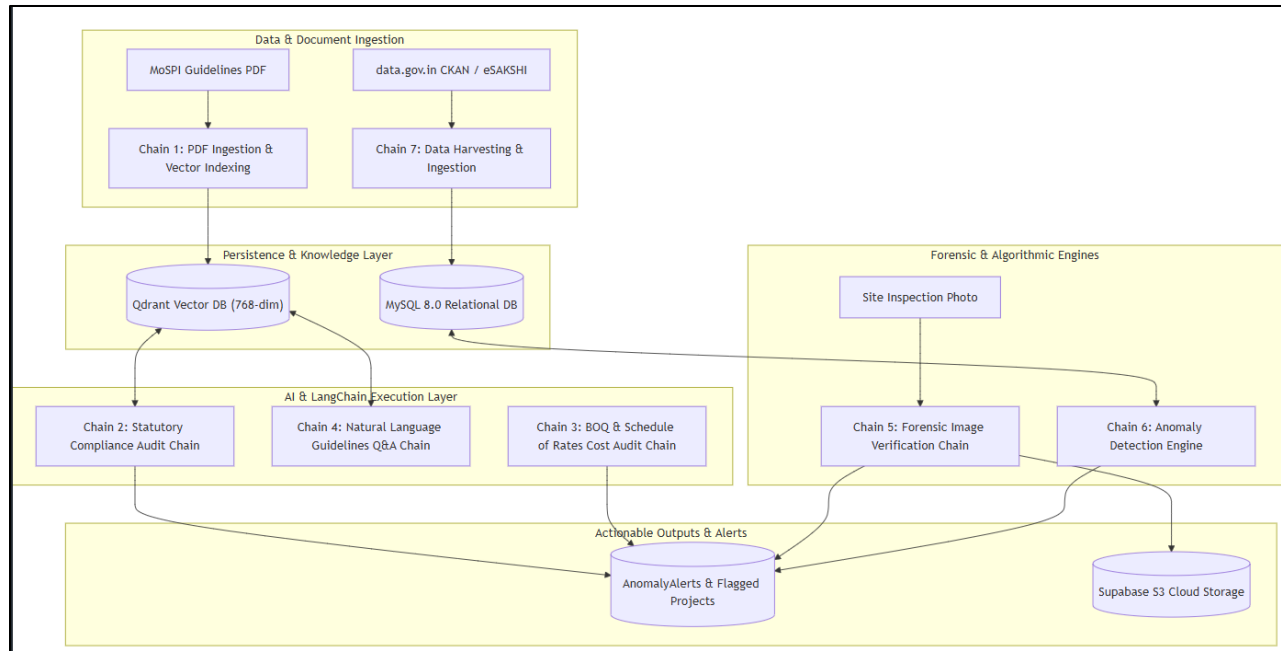
We went with React mainly because we knew from the start that we'd need multiple dashboards — Ministry Admin, District Authority, MP, Contractor, and a public citizen view — that all share the same building blocks (charts, GIS maps, proposal tables) but show different data to different people. React's component model let us build those pieces once and reuse them everywhere, and its virtual DOM means a single incoming alert or a contractor's photo upload only re-renders the part of the screen that actually changed, not the whole dashboard.

Vite won out over older tooling like Create React App mostly because of how it handles development. It serves source files on demand as native ES modules, so the dev server starts almost instantly and Hot Module Replacement stays fast no matter how big the project gets — and under the hood it pre-bundles dependencies with esbuild, a Go-based tool that's roughly 10 to 100 times faster than the older JavaScript bundlers, before handing production builds off to Roll up for tree-shaking and optimisation. This mattered for us in a very practical way too: because Vite is so light, the dev machine's RAM and GPU stay free for the heavier local AI work — the Dockerized models, the Qdrant vector database, FastAPI's own processing — instead of getting eaten up by frontend tooling.

05

Backend Engineering: Core Pipelines

Rather than one monolithic service trying to do everything, we split the backend into five pipelines that each do one job — ingestion, compliance checking, anomaly detection, and forensic verification can all be built, tested, and scaled on their own timelines.



Pipeline A — Data Ingestion & Harvesting

This lives in `extractor.py` and runs as part of the app's startup sequence. A CKAN harvester queries `data.gov.in` for sanctioned works, district codes, budgets, and expenditure; an e-SAKSHI scraper polls for real-time sanction status, instalment releases, and utilisation certificates; and if either upstream source is unreachable or rate-limited, the system falls back to a bundled dataset of realistic MPLADS projects spanning several states, so a demo or a development environment never sits there with an empty database just because a government API happened to be down. On startup, the app also checks whether the relational database is empty and, if so, seeds it automatically with projects, registered contractors, site inspections, and a handful of sample alerts.

Pipeline B — MoSPI Statutory Guidelines Vector RAG

```
MoSPI Guidelines PDF → PyPDF extraction → recursive chunking (1000/200) →
Gemini embeddings (models/gemini-embedding-001) → Qdrant collection "mplads_guidelines"
```

The 2023 MoSPI guidelines get split into 1,000-character chunks with a 200-character overlap, so we don't accidentally cut a clause in half at a chunk boundary. Each chunk is embedded with Google's `gemini-embedding-001` model and stored in Qdrant under cosine similarity. When someone audits a project proposal, its title, category, description, and budget get turned into a query embedding, the top-k closest statutory clauses ($k=4-6$, depending on the endpoint) come back from Qdrant, and Gemini 3.6 Flash takes it from there — returning a structured judgement on whether the work is even permissible (flagging things like religious structures or private-property assets, which MPLADS explicitly disallows), whether the cost fits within the state Schedule of Rates, and the exact chapter and clause backing up that judgement.

Pipeline C — Statistical & Geospatial Anomaly Engine

This is the part that runs on its own, without anyone needing to click a button:

- SC/ST quota audit (Clause 2.3): flags a Member of Parliament's constituency if SC-area expenditure falls below 15% or ST-area expenditure falls below 7.5% of total annual expenditure.
- Cost-inflation check: flags a work whose cost exceeds the configured benchmark by more than a 15% tolerance.
- Cartelization check: computes each contractor's market share of awarded value in a district; a share above 30% raises a monopoly warning.
- Tender-splitting detection: flags two contracts awarded to the same contractor within 5 km of each other where both individual amounts fall under the ₹5,00,000 e-tendering threshold — a pattern consistent with deliberately splitting a larger work to dodge competitive tendering.
- Duplicate-asset detection: uses the Haversine great-circle distance formula to flag a newly proposed asset of the same category within 50 metres of an existing asset.
- Progress-expenditure mismatch: flags a project where financial progress outruns physical progress by more than 25% (MEDIUM), escalating to HIGH once that gap passes 40%.
- SLA breach detector: flags projects where more than 45 days have elapsed between MP recommendation and administrative sanction; the alert is auto-escalated from HIGH to CRITICAL once the gap exceeds 90 days.

Every one of these checks reads from a set of statutory constants defined in one place, rather than hard-coded separately inside each function — so if a guideline changes, we update a single number, not five different files:

Constant	Value
Annual entitlement per MP	₹5 crore
Maximum spend outside home constituency	₹50 lakh
Project completion limit	1 year from sanction
Sanction-processing SLA	45 days (Clause 4.2)
SC-area allocation floor	15% of annual expenditure
ST-area allocation floor	7.5% of annual expenditure
Cost-inflation tolerance	15% above benchmark
Cartelization threshold	30% contract-value share

Pipeline D — Computer Vision & Forensic Image Verification

```
Raw Photo → EXIF stream analysis (camera + editing-software tags)
           → GPS EXIF extraction (DMS → decimal) → Haversine vs. sanctioned centroid (≤100m)
           → EXIF timestamp vs. server upload time → weighted tamper-risk score (0–1)
```

Every progress photo a contractor uploads gets checked for editing-software fingerprints in its EXIF Software tag — Photoshop, GIMP, Canva, Lightroom, Capture One, Snapseed, Pixlr, Fotor, BeFunky, Paint.NET, Affinity, Corel, PicMonkey, and Adobe are all on the list — along with generic giveaway words like "edited", "modified", or "retouched" showing up anywhere in the tags. We pull the GPS coordinates out of the EXIF data, convert them from degrees-minutes-seconds to plain decimal, and run them through the Haversine formula against the project's sanctioned centroid; anything more than 100 metres off gets marked as a geo-mismatch. All of this rolls up into one tamper-risk score capped at 1.0 — a matched editing tool adds 0.4, a suspicious keyword adds 0.2, no timestamp adds 0.2, missing camera info (which usually means a screenshot or a downloaded image) adds 0.3, and no GPS at all — which eSAKSHI actually requires for site photos — adds another 0.3. We also compare the photo's own capture timestamp against when it was uploaded, mainly to catch someone resubmitting an old photo from a previous inspection.

50m / 100m / 45 days

The three statutory tolerances the anomaly engine checks automatically: duplicate-asset proximity, geotag displacement, and sanction-processing SLA.

Directory Map

```
backend/
├── main.py           # entry point, CORS, startup lifespan, DB/Qdrant health
├── config.py         # Pydantic Settings – statutory constants & thresholds
├── models.py         # ORM: User, Project, Contractor, Contract Award,
                    #   SiteInspectionPhoto, Anomaly Alert
├── schemas.py        # Pydantic request/response schemas
├── routers/
│   ├── auth.py       # login, JWT issuance, RBAC dependency
│   ├── projects.py   # CRUD, geo-bounding, category filters
│   ├── audit.py      # RAG compliance triggers, manual audit workflow
│   └── reports.py     # aggregated dashboard statistics (Ministry view)
├── services/
│   ├── extractor.py  # CKAN / e-SAKSHI harvester
│   ├── llm_api.py    # Gemini RAG + Qdrant retrieval
│   ├── anomaly_engine.py # SC/ST, cartelization, delay, duplicate checks
│   ├── image_verifier.py # EXIF geotagging & tamper-risk scoring
│   └── storage.py     # Supabase photo upload, local-disk fallback
```

06

Risk Scoring Framework

Weighted Composite Score

Every work ends up with one risk score between 0 and 100, built from seven sub-scores that each carry a different weight depending on how reliably — and how seriously — they tend to point at real irregularity:

Risk Factor	Weight	What it Measures
Financial Anomaly Score	25%	Deviation of cost/expenditure from the cohort benchmark
Physical–Financial Mismatch	20%	Gap between fund utilisation and verified on-ground progress
Delay Probability	15%	Likelihood of missing the approved completion schedule
Cost Overrun Score	15%	Deviation between sanctioned and revised/actual cost
Duplicate-Work Score	10%	Strength of evidence that a work duplicates another sanctioned work
Document Compliance Score	10%	Completeness and consistency of mandatory documentation
Geo/Image Verification Score	5%	Consistency of photographic and location evidence

Risk Bands

Score Range	Risk Level	Recommended Action
0 – 24	Low	Routine monitoring; no special action required
25 – 49	Moderate	Desk review of documents and progress reports
50 – 74	High	Priority field inspection within a defined SLA
75 – 100	Critical	Immediate escalation for investigation

Explainability

We didn't want to hand someone a number and expect them to trust it, so every score comes with a plain-language explanation of what actually drove it — something like: "High risk: expenditure is 42% above the benchmark for similar works, physical progress is 18% below financial progress, and the same implementing agency has three delayed projects." If an official is going to act on a flag, they should be able to see exactly what put it there before they do anything about it.

Shipped vs. designed

The reporting form, issue taxonomy, and ticket reference already run in the public portal. The routing, freeze, and escalation automation below is the defined next build phase on top of the existing Anomaly Alert model.

Where a Report Is Designed to Go

Here's the idea we're building toward: a citizen files a report, and it goes to three places at once. It lands in the District Authority's inspection queue, since the District Magistrate is who's actually responsible for physical inspection and show-cause notices under MPLADS guidelines. It also shows up in the Ministry's fraud and systemic-risk feed, so the Central Audit Wing can spot patterns across districts — a contractor showing up too often, say, or a cluster of ghost projects. And it gets checked automatically against the anomaly engine's own data on that project: does the reported location actually match the sanctioned GPS point, and does the contractor's claimed progress ("superstructure 60% complete") line up with what the citizen is describing (work that, as far as they can tell, was never started).

Issue Categories and Statutory Risk

This part is already live — the public-portal form tags every submission with one of the categories below and hands back a ticket reference (something like GRV-2026-XXXXXX) that the citizen can quote later.

Category	Description	Statutory Risk Level
WORK_NOT_FOUND	No physical work on ground (ghost project)	Critical — immediate disbursal freeze
PRIVATE_ASSET	Work executed on private property (inadmissible)	Critical — Guideline Section 3.2 violation
SUBSTANDARD	Substandard construction quality / materials	High — withhold milestone payment
TAMPERED_BOARD	Missing / misleading citizen information board	Medium — notice to contractor
DELAY_ABANDONED	Project abandoned by contractor	High — contract termination / penalty
OTHER	Cost inflation, schedule tampering, cartelization	Medium / High — audit review

Designed Resolution Workflow

Once the grievance router is actually wired into the existing alerting infrastructure, this is the path a validated report would follow:

1. Flagging & triage — the anomaly engine raises a CRITICAL-severity alert on the project, and a warning banner shows up wherever that project appears, public or internal.
2. Financial hold — a Ministry auditor or the District Collector freezes the milestone fund, so nothing gets disbursed to the contractor while the inquiry is open.
3. Ground inspection — the District Collector sends out an SDM or Executive Engineer, who runs the same EXIF/GPS verification tool we already have in the Site Inspections Queue against the contractor's claims.
4. Show-cause & escalation — if the complaint checks out, the District Authority issues a show-cause notice, and the Ministry can push it up for a formal statutory inquiry.
5. Resolution — if the contractors at fault, they get blacklisted, fund recovery kicks in, and the work goes back out to tender. The underlying alert simply moves from OPEN to INVESTIGATING to RESOLVED, using the same status lifecycle the anomaly engine already relies on.

08

Multi-Citizen Aggregation & Escalation

Here's what we want to build next: when several citizens report the same project, the system shouldn't treat those as five unrelated tickets sitting in five different queues. Each person would still get their own reference number so they can track things themselves, but behind the scenes, any complaint matching the same project — or landing within Haversine range of the same coordinates — would get pulled into one shared incident record, with photos from different people at different times stitched into a single timeline for whoever ends up investigating.

Planned Severity Escalation

Report Volume	Target Severity	Planned Action
1 citizen report	Medium	Added to routine district scrutiny queue
3 – 5 citizen reports	High	Auto-escalated to District Collector; priority inspection dispatched
6+ citizen reports	Critical	Emergency freeze — disbursements blocked, Central Audit alerted

The plan is for each corroborated complaint to bump up the contractor's `risk_score` — a field that already exists on the Contractor model — so a cluster of reports on one project automatically puts every other active project from that same contractor under a bit more scrutiny.

Planned Financial Safeguards and Anti-Spam Controls

When reports pile up around the more serious categories — `WORK_NOT_FOUND`, `PRIVATE_ASSET` — the design has the project status flip automatically from `IN_PROGRESS` to `FLAGGED_REVIEW` (a state

ProjectStatus already supports), locking milestone invoices in the contractor portal until someone clears it. And because any system that lets the public flag contractors is also a system a rival could try to game, we're planning basic rate-limiting on repeated submissions from the same source, perceptual image-hash matching to catch the same photo getting re-uploaded under different names, and giving more weight to reports that actually come from within the project's own district.

Unified Resolution

Once an officer finishes the site inspection, one inquiry report should be able to close the whole cluster at once — every citizen who reported that project gets notified together, and the project's public card carries a visible audit-inquiry badge for as long as the case stays open.

09

Infrastructure & Deployment

We containerised everything with Docker mainly so nobody has to hand-install and version-match Python 3.11, Node.js 20, MySQL 8.0, Qdrant, and OpenCV's system dependencies on their own machine. One docker-compose.yml brings the whole system up with a single command — it keeps the OS layers consistent, wires every service together over an internal network so containers can find each other by name, waits for MySQL to actually report healthy before starting the backend, and keeps MySQL and Qdrant data in named volumes so a container restart doesn't wipe anything out.

Service	Container	Base Image	Ports	Purpose
MySQL	mplad_mysql	mysql:8.0	3306	Relational store — users, sanctions, contracts, alerts
Qdrant	mplad_qdrant	qdrant/qdrant: latest	6333, 6334	Vector store for RAG guideline embeddings (cosine similarity)
Backend	mplad_backend	python:3.11-slim (custom)	8000	FastAPI, RAG audit, anomaly engine, EXIF/image forensics, auth
Frontend	mplad_frontend	node:20-alpine (custom)	5173	React/Vite UI for all role-based portals

Photo Storage

Inspection photos get uploaded to a Supabase Storage bucket over a signed REST call, which gives every image a durable, public CDN URL without us having to run a separate object-storage container. If Supabase credentials aren't set — say, in an offline or air-gapped deployment — the storage service just falls back to local disk on its own, so uploads keep working without touching a line of code.

```
docker compose up --build      # start everything, building images
docker compose up -d          # start in the background
docker compose down           # stop everything
docker compose down -v        # stop and wipe database volumes
```

10

Security, RBAC & Governance

Role-Based Access Control

Four roles get authenticated access, each scoped to what they actually need to do. Citizens are deliberately left outside that login boundary — the public portal and the reporting form are meant to be open, so filing a report never means creating an account or sitting through a login screen first:

Role	Scope
ministry_admin	National oversight, contractor blacklisting, cross-state quota analytics
district_authority	Sanction management, inspection audits, field-officer dispatch
mp	Constituency expenditure tracking, SC/ST quota metrics
contractor	Work-order views, camera-only photo uploads with instant EXIF validation
citizen (unauthenticated)	Public transparency portal, constituency map exploration, grievance filing

Authentication runs on signed JWT tokens carrying subject, role, and expiry claims. Protected endpoints check the role through FastAPI's dependency injection, so a request that isn't authorised for a given handler gets stopped at the routing layer — before any business logic even runs.

Platform-Wide Controls

- Encryption of data in transit and at rest, behind a secure API gateway.
- Multi-factor authentication for administrative and investigative roles, as the platform matures beyond the current password-based login.
- Audit logging on access to sensitive or investigative data.
- Data masking for sensitive fields where a role doesn't need full visibility.
- A persistent `project_uid` that follows a work from recommendation all the way to asset creation — already the key every alert, photo, and contract award in the schema is linked to.

11

Stakeholder Dashboards

Stakeholder	Primary Dashboard Content	Typical Decision Supported
Ministry Admin	National fund utilisation, state/district comparison, fraud-risk intelligence feed, contractor blacklisting	Policy-level oversight and escalation
District Authority	Operational workbench, inspection queue, financial vs. physical progress, sanction management	Day-to-day operational monitoring
Member of Parliament	Recommended/sanctioned/completed works, fund utilisation, SC/ST quota metrics	Constituency-level self-monitoring
Contractor	Work orders, camera-only site photo upload with instant EXIF validation	Progress reporting and milestone claims
Citizen	Public transparency portal, constituency map, grievance filing and tracking	Community-level accountability

12

Implementation Roadmap

Phase	Focus	Key Deliverables
1: Discovery & Data Assessment	Understand existing data and workflows	Data-source inventory, quality assessment, fraud-indicator definitions
2: Minimum Viable Platform	Core ingestion and dashboards	Ingestion pipelines, baseline dashboards, rule-based checks, basic risk scoring
3: Advanced AI Features	Full detection capability	Anomaly detection, duplicate-work detection, RAG compliance, image forensics
4: Pilot Deployment	Real-world validation	Selected districts/states, false-positive measurement, workflow refinement
5: National Rollout	Scale and institutionalise	All-India expansion, official training, operating procedures

13

Success Metrics

- How much manual monitoring effort drops per work reviewed.
- What proportion of high-risk works turn out to be genuine on inspection.
- The false-positive rate on alerts, tracked over time as the thresholds get tuned.
- Improvement in fund-utilisation rates at the district and state level.
- Fewer confirmed duplicate works and misreported assets over time.
- How long it takes, on average, to go from a grievance being filed to it being resolved — once the grievance backend is built out.
- How much each role actually uses their dashboard — login frequency for Ministry, District, MP, and Contractor; visits and reports filed for the public citizen portal.

14

Conclusion & Future Scope

We set out to build a decision-support and early-warning platform, not something that hands down verdicts on its own. MPLADS Rakshak brings together rule-based compliance checks, statistical and geospatial anomaly detection, a retrieval-augmented guideline engine, and — once fully built out — a citizen reporting layer that clusters and escalates on its own. But every single output stays a recommendation. The District Collector, the State Nodal Authority, and the Ministry still make the call. What changes is that MPLADS oversight stops being a once-in-a-while audit exercise and starts looking more like continuous, risk-prioritised monitoring, without taking that decision-making authority away from the people who are actually supposed to hold it.

Looking ahead, there's a fair bit we'd like to add: pulling in data from other central and state schemes to catch duplicate assets that hide across scheme boundaries, layering in satellite-imagery change detection so asset creation can be verified independently of whatever photo a contractor submits, and building out the citizen portal so people can weigh in on a project's status well after their initial report, not just once.